



ICPC - VNUHCM University of Science contest

Date: October 4th, 2026

OVERVIEW

Problem A: Danh the Naughty Pig	1
Problem B: Freezer	3
Problem C: Fluorine's Fun Function	5
Problem D: Tidal Wave	6
Problem E: Danh the Pig Emperor	8
Problem F: Wagon Sorting	10
Problem G: Whisper Chain	11
Problem H: Raining	12
Problem I: Danh the Happy Pig	14
Problem J: Crafting Costs	15
Problem K: Lithium and Lithuania	17
Problem L: Lucky Numbers	19



Problem A

Danh the Naughty Pig

Time limit: 3s

Memory limit: 1GB

You are given an $n \times n$ playground grid. Some cells on the grid contain obstacles. Let S be the set of these obstacle cells.

Danh, the naughty pig, loves to bounce around the grid! His goal is to bounce on every single obstacle in the playground. However, since his energy is limited, he can only play (start a new bouncing sequence) at most n times.

Your task is to help Danh partition the set of obstacles S into k non-empty bouncing sequences ($k \leq n$) such that each sequence $c_1, c_2, \dots, c_l$ satisfies his naughty bouncing rules:

- for every i ($1 \leq i < l$), the bounce from cell c_i to c_{i+1} must share a row or share a column (they do not need to be strictly adjacent);
- for every i ($1 \leq i \leq l - 2$), Danh refuses to bounce in the same direction twice in a row. Thus, the segments $c_i c_{i+1}$ and $c_{i+1} c_{i+2}$ must be perpendicular (meaning one bounce is horizontal and the next is vertical, or vice versa).

Note that a sequence of length 1 is always trivially valid, and a sequence of length 2 is valid as long as it satisfies the first condition.

It can be shown that a valid partition with $k \leq n$ sequences always exists. Find any such partition so Danh can successfully bounce on all the obstacles!

Input

- The first line contains two integers n and m ($1 \leq n \leq 2 \times 10^5$, $0 \leq m \leq \min(n^2, 2 \times 10^5)$) — the size of the playground grid and the number of obstacles.
- Each of the next m lines contains two integers r_i and c_i ($1 \leq r_i, c_i \leq n$) — the row and column coordinates of an obstacle cell. It is guaranteed that all given cells are distinct.

Output

In the first line, print a single integer k ($0 \leq k \leq n$) — the number of plays (sequences) Danh will make. Then, print the descriptions of the k bouncing sequences. For each sequence, print its length l on one line, followed by l lines containing the coordinates of the cells in the sequence, in the exact order Danh bounces on them.

Each obstacle cell from S must appear in exactly one sequence. If there are multiple valid partitions satisfying $k \leq n$, you may output any of them.



Sample Input	Sample Output
2 3 1 1 1 2 2 2	1 3 2 2 1 2 1 1
8 12 2 2 2 5 5 5 5 7 7 7 7 3 3 3 3 1 1 8 4 8 8 4 8 8	3 1 4 8 3 1 8 8 8 8 4 8 3 1 3 3 7 3 7 7 5 7 5 5 2 5 2 2

$n \times n$ grid m cell

- $\leftrightarrow$ same row/column
- không được nhảy cùng row/column giữa 2 lần

In ra 1 cách nhảy hoàn bộ cell

- số lần nhảy
- số step
- ô cần nhảy



Problem B

Freezer

Time limit: 1s

Memory limit: 256MB

A blizzard has cut Khanh's polar research station off from the rest of the world. Until the storm is over, the food in the station's freezer is all Khanh has, so it must be rationed carefully.

The freezer holds n food portions, numbered from 1 to n . Portion i has an expiration day e_i and a quality score q_i . Days are numbered $1, 2, 3, \dots$, and portion i stays edible until the end of day e_i , so it can be eaten on any of the days $1, 2, \dots, e_i$.

Every day Khanh eats exactly one meal, which consists of two different portions. A meal eaten on day d is acceptable only if all of the following hold:

- both portions are still edible on day d , that is, the expiration day of each of them is at least d ;
- each of the two portions has quality at least 4; $\nearrow$
- the total quality of the two portions is at least 9. $\nearrow$

Each portion can be eaten at most once.

Find the largest K such that Khanh can eat an acceptable meal on each of the days $1, 2, \dots, K$, and give a plan of meals for these days.

Input

- The first line contains a single integer n ($1 \leq n \leq 2000$) — the number of portions. $n^2 \log$
- The i -th of the next n lines contains two integers e_i and q_i ($1 \leq e_i \leq 2000$, $1 \leq q_i \leq 10$) — the expiration day and the quality of portion i .

Output

On the first line, print K , the maximum number of days. Then print K lines. The d -th of them must contain two distinct integers, the indices of the two portions eaten on day d , in any order. If $K = 0$, print only the first line.

If there are several optimal plans, print any of them.



Sample Input	Sample Output
7 3 4 1 6 2 9 5 4 2 3 4 5 1 8	3 7 2 1 3 4 6
5 10 4 10 4 10 4 10 7 1 10	2 2 5 3 4

Notes

In the first example, portion 5 has quality 3 and can never be eaten, so at most $6/2 = 3$ days can be covered. The plan above uses portions 7 and 2 on day 1, portions 1 and 3 on day 2, and portions 4 and 6 on day 3. The last meal has total quality $4 + 5 = 9$, which is just enough.

In the second example, two portions of quality 4 do not form an acceptable meal, because $4 + 4 < 9$. So every meal contains portion 4 or portion 5, and at most two days can be covered. Portion 5 can only be eaten on day 1, so in a two-day plan it is part of the first meal.



Problem C

Fluorine's Fun Function

Time limit: 2s
Memory limit: 1GB

Fluorine just learned about the Fibonacci sequence today, and he is excited to share with everybody! The Fibonacci sequence is a famous series of numbers where each number is the sum of the two preceding ones, starting with two 1s. Here are some of the first few terms of the Fibonacci sequence:

1, 1, 2, 3, 5, 8, 13, 21, ...

Fluorine thinks this sequence is really cool, and he made his so-called fun function: $F(i)$, which is the i -th number of the Fibonacci sequence.

For instance: $F(1) = F(2) = 1$, $F(3) = 2$, $F(4) = 3$.

Fluorine told his idea to his sister, Francium, who is a computer science student, and she find Fluorine's fun function intriguing too. This is why she came up with a programming problem:

You are given an array a of length n consisting of positive integers. You need to process q queries on this array. There are two types of queries:

- 1 $l\ r\ x$: Add x to a_i for all i such that $l \leq i \leq r$.
- 2 $l\ r$: Calculate the sum of the Fibonacci numbers corresponding to the elements in the subarray from l to r . Specifically, compute $\sum_{i=l}^r F(a_i) \pmod{10^9 + 7}$.

Because Fluorine is no coder, he cannot solve his sister problem. Can you help him?

Input

- The first line contains two integers n and q ($1 \leq n, q \leq 2 \cdot 10^5$) — the size of the array and the number of queries.
- The second line contains n positive integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the initial elements of the array a .
- Each of the next q lines contains a query in one of the following formats:
 - 1 $l\ r\ x$ ($1 \leq l \leq r \leq n$, $-10^9 \leq x \leq 10^9$)
 - 2 $l\ r$ ($1 \leq l \leq r \leq n$)

1 2 3 4

It is guaranteed that the value of a_i , for all $1 \leq i \leq n$ at all time is a positive integer.

Output

For each query of type 2, print a single line containing the answer modulo $10^9 + 7$.

Sample Input	Sample Output
4 4	7
1 2 3 4	11
2 1 4	9
1 2 3 2	
2 2 4	
2 1 3	



Problem D

Tidal Wave

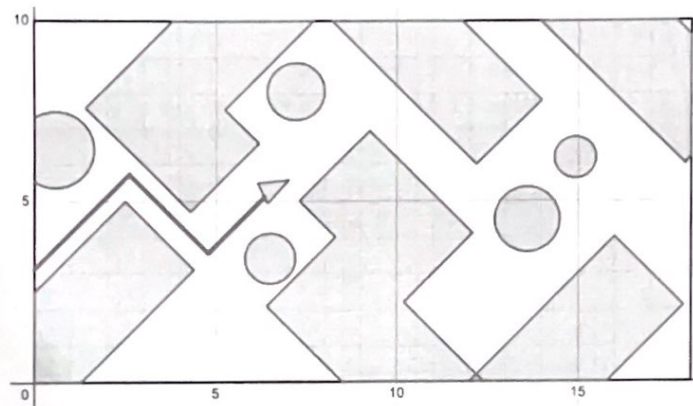
Time limit: 1s

Memory limit: 1GB

I want to be your tidal wave.

— Someone

Geometry Dash is a fast-paced, rhythm-based platformer game developed by RobTop Games, where players control a geometric character through obstacle courses timed to electronic music. Though the game looks deceptively easy at first glance, its skill ceiling can get ridiculously high, with top levels requiring hundreds of pixel-perfect inputs to beat.



A redraw of the original level for illustration only; it does not represent a problem input or the exact model used below.

A top player named Zoink wants to prove to the world that he can conquer a supposedly impossible level named *Tidal Wave*. But there is a problem: because no one had dared to even play the level before him, *Tidal Wave* is poorly tested and notoriously unfun to play! Therefore, Zoink took matters into his own hands and decided to *balance* the level's gameplay himself.

However, modifying gameplay is a tricky task, and Zoink might accidentally make the level mathematically impossible. He needs your help to develop an algorithm to verify if his renovated level can actually be beaten, saving him from having to playtest it manually.

A level can be modeled as a grid with n rows and m columns. The cell in the i -th row from the top and the j -th column from the left is denoted by (i, j) .

The character is initially located at $(1, 1)$ and wants to reach any cell in the m -th column. From a cell (i, j) , the character can move to either $(i + 1, j + 1)$ or $(i - 1, j + 1)$. The character cannot move outside the boundaries of the grid or enter a blocked cell.

For each column j , its topmost l_j cells and bottommost r_j cells are blocked. More precisely, a cell (i, j) is blocked if $1 \leq i \leq l_j$ or $n - r_j + 1 \leq i \leq n$. If $l_j = 0$, no cells are blocked from the top; if $r_j = 0$, no cells are blocked from the bottom. In particular, if $l_j = r_j = 0$, all cells in the column are unblocked.

Determine whether the character can successfully reach the m -th column.



Input

- The first line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.
- For each test case:
 - The first line contains two integers n and m ($1 \leq n, m \leq 10^5$) — the number of rows and columns of the grid.
 - Each of the next m lines contains two integers l_j and r_j ($0 \leq l_j, r_j \leq n$, $l_j + r_j \leq n$) — the number of blocked cells at the top and bottom of the j -th column.

It is guaranteed that the starting cell $(1, 1)$ is not blocked, meaning $l_1 = 0$ and $r_1 < n$. It is also guaranteed that the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print YES if the character can reach the m -th column, otherwise print NO.

You can output the answer in any case (upper or lower). For example, the strings YES, yes, yEs, and Yes will be recognized as positive responses, and the strings NO, no, No will be recognized as negative responses.

Sample Input	Sample Output
3 1 1) 0 0) 3 3) 0 0) 0 3) 1 0) 6 9) 0 4) 0 3) 1 2) 2 1) 1 2) 0 3) 1 2) 2 1) 3 0)	YES NO YES

Notes

In the first test case, the character starts in the last column, so the answer is YES.

In the second test case, every cell in the second column is blocked, so the character cannot reach the last column.

In the third test case, one valid path visits rows 1, 2, 3, 4, 3, 2, 3, 4, 5 in columns 1, 2, ..., 9, respectively.



Problem E

Danh the Pig Emperor

Time limit: 1s
Memory limit: 1GB

Max total area is 10^{24}

— _Satoru

In the glorious ancient empire ruled by Danli, the Pig Emperor, his loyal subjects have decreed the construction of N magnificent cardboard pyramids upon the great 2D Cartesian plains (Oxy).

By royal architectural design, each pyramid is shaped as an isosceles right triangle. Its base (the hypotenuse) rests perfectly upon the sacred X-axis, and its peak (the right angle) points skyward into the positive Y half-plane ($y > 0$).

You, the Imperial Scholar, are given the ancient scrolls describing these N pyramids. The base of the i -th pyramid spans the segment from $x = l_i$ to $x = r_i$ on the X-axis. Note that by the Emperor's grand and chaotic vision, these monuments may overlap or completely encompass one another in any arbitrary manner.

Your task is to determine the total area of the union of all N pyramids. In other words, calculate the total area of the region on the plane that is occupied by at least one of the Emperor's cardboard monuments.

Input

- The first line contains a single integer N ($1 \leq N \leq 2 \times 10^5$) — the number of cardboard pyramids decreed by the Pig Emperor.
- Each of the next N lines contains two integers l_i and r_i ($-10^9 \leq l_i < r_i \leq 10^9$) — the coordinates of the endpoints of the i -th pyramid's base on the X-axis.

Output

Output a single integer — the total area of the union of the N pyramids multiplied by four. It is guaranteed that this value is an integer.

Sample Input	Sample Output
2 0 4 2 6	28

Notes

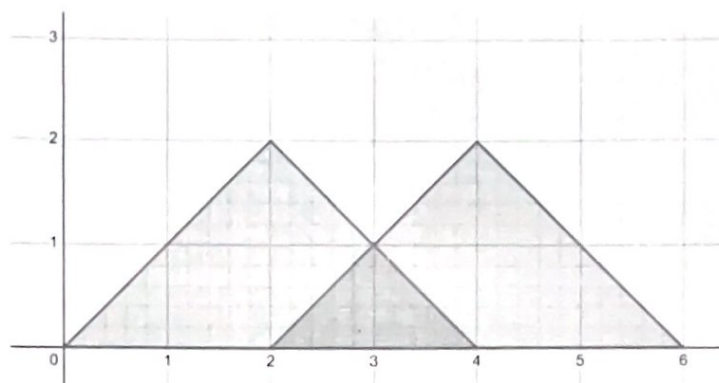


Figure 1: The formation of pyramid as shown in the example. It can be shown that the area of the union of the pyramids is 7.

**Problem F****Wagon Sorting**

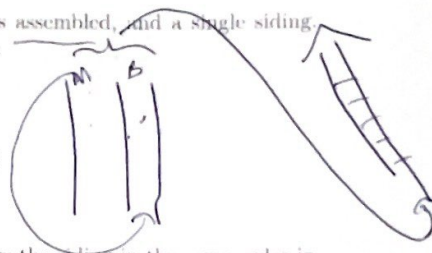
Time limit: 1s

Memory limit: 256MB

Mr. Tuan is the foreman of a small freight yard. This morning N wagons stand in a row on the main track. Every wagon carries a label from 1 to N that tells its place in the train Mr. Tuan has to assemble: the wagon labeled 1 must be at the front of the train, the wagon labeled 2 right behind it, and so on. No two wagons have the same label.

Besides the main track, the yard has a train track, where the train is assembled, and a single siding. Wagons are moved one at a time, and only in the following three ways:

1. move the first wagon of the main track to the back of the train;
2. move the first wagon of the main track to the back of the siding;
3. move the first wagon of the siding to the back of the train.



Thus wagons leave the main track in their original order, and they leave the siding in the same order in which they entered it. A wagon that has been put on the train stays there.

Mr. Tuan suspects that the whole train cannot always be assembled this way, so he wants to know how far he can get. Find the largest integer M such that some sequence of moves makes the train consist of exactly the wagons labeled $1, 2, \dots, M$, in this order from front to back. The other wagons may stay on the main track or on the siding, but none of them may be put on the train.

Input

- The first line contains a single integer N ($3 \leq N \leq 100\,000$) — the number of wagons.
- The second line contains N distinct integers $S_1, S_2, \dots, S_N$ ($1 \leq S_i \leq N$) — the labels of the wagons on the main track, from the first wagon to the last. The wagon labeled S_1 is the first one to leave the main track.

ma t
b t
m t

Output

1 4 1 2 6 3 5

Print a single integer — the largest possible value of M .

Sample Input	Sample Output
6 3 1 5 4 2 6	3
5 2 4 1 3 5	5

(5)
5
4
6

Notes

In the first example, Mr. Tuan moves wagon 3 to the siding, wagon 1 to the train, wagons 5 and 4 to the siding, wagon 2 to the train, and then wagon 3 from the siding to the train. Now wagon 5 is at the front of the siding and wagon 4 is behind it, so wagon 4 can never reach the train before wagon 5. No sequence of moves does better, so the answer is 3.

In the second example, all five wagons can be put on the train in the right order.

1
3
4

3
1
2
2



Problem G

Whisper Chain

Time limit: 1s
Memory limit: 256MB

Ms. Hoa's class of n students is playing the whisper game. The students sit on n chairs placed in a row and numbered from 1 to n . The student on chair 1 whispers a message to the student on chair 2, who passes it on to the student on chair 3, and so on, until the message reaches the student on chair n .

The students want the message to reach the end of the row unchanged, so each of them has named the classmate they would like to whisper to. The students are numbered from 1 to n , and student i has named student s_i . The wish of student i is fulfilled if and only if student s_i sits on the chair right after the chair of student i , that is, student i sits on some chair k and student s_i sits on chair $k + 1$. In particular, the wish of the student on chair n is never fulfilled.

Ms. Hoa wants to seat the students so that the number of fulfilled wishes is as large as possible. Find such a seating.

Input

- The first line contains a single integer n ($5 \leq n \leq 10\,000$), the number of students.
- The second line contains n integers $s_1, s_2, \dots, s_n$ ($1 \leq s_i \leq n$, $s_i \neq i$), where s_i is the student whom student i wants to whisper to.

Output

On the first line, print a single integer M , the largest possible number of fulfilled wishes.

On the second line, print n distinct integers $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq n$), where c_i is the number of the chair on which student i sits. Exactly M wishes must be fulfilled in this seating.

If there are several optimal seatings, you may print any of them.

Sample Input	Sample Output
9 9 3 7 8 9 4 2 6 3	6 1 5 3 7 6 9 4 8 2
5 3 4 5 1 2	4 1 4 2 5 3

Notes

In the first example, chairs 1 to 9 are taken by students 1, 9, 3, 7, 2, 5, 4, 8, 6, in this order. The wishes of students 1, 9, 3, 7, 4 and 8 are fulfilled. It can be shown that no seating fulfills 7 wishes.

In the second example, chairs 1 to 5 are taken by students 1, 3, 5, 2, 4, and every wish except the one of student 4 is fulfilled. Remember that the output lists the chair of each student, not the student on each chair.



Problem H

Raining

Time limit: 1s

Memory limit: 1GB

What the * is convex combination?

— Steveonalex

Please read the input format carefully.

City X occupies the square region $[-10^9, 10^9] \times [-10^9, 10^9]$.

Demen, Natsupercell, and Steveonalex are meteorologists in City X. While observing an unusually heavy downpour, they use a machine to determine which locations in the city are experiencing rain.

After q seconds, the meteorologists have received reports of q points $p_1, p_2, \dots, p_q$, where p_i is the point at which rain was reported to have started during the i -th second. All reported points are distinct.

According to Demen, after the t -th second, a point is raining if and only if it is a convex combination of the t points reported so far. More precisely, a point P is raining if there exist real numbers $\alpha_1, \alpha_2, \dots, \alpha_t$ such that

- $\alpha_i \geq 0$ for every $1 \leq i \leq t$;
- $\sum_{i=1}^t \alpha_i = 1$;
- $\sum_{i=1}^t \alpha_i p_i = P$.

Assuming that Demen is correct (as he always is), the meteorologists estimate the damage caused by the downpour as the maximum possible value of twice the area of a triangle whose three vertices are raining points.

For each second $1, 2, \dots, q$, determine the estimated damage.

Input

- The first line contains a single integer q ($1 \leq q \leq 10^5$), the number of reports made by the machine.
- Each of the next q lines contains two integers x_i and y_i ($-10^9 \leq x_i, y_i \leq 10^9$), indicating that rain was reported to have started at point $p_i = (x_i, y_i)$ during the i -th second.

Note: The points $p_1, p_2, \dots, p_q$ are generated uniformly at random without replacement from the set of points with integer coordinates in $[-10^9, 10^9] \times [-10^9, 10^9]$. Therefore, all reported points are distinct.

Output

Print q lines. The i -th line should contain the estimated damage after the i -th second.

Note: Three collinear points form a degenerate triangle with area 0.



Sample Input	Sample Output
10	0
0 0	0
-3 5	41
4 7	41
1 9	93
-10 3	150
0 15	152
1 1	154
2 2	240
10 3	360
0 -15	

Notes

Note: The example test is not generated randomly, but your code still has to run correctly on this test.

After the first and second seconds, the raining region is a single point and a line segment, respectively, so both answers are 0.

After the third second, the raining region is the triangle with vertices $(0, 0)$, $(-3, 5)$, and $(4, 7)$. Its doubled area is $|(-3) \cdot 7 - 5 \cdot 4| = 41$.

After the tenth second, one maximum-area triangle has vertices $(-10, 3)$, $(10, 3)$, and $(0, -15)$. Its base has length 20 and its height is 18, so the answer is $20 \cdot 18 = 360$.



Problem I

Danh the Happy Pig

Time limit: 2s

Memory limit: 1GB

Danh is a happy pig currently standing at cell 0 on a 1-dimensional board. The board consists of cells numbered from 0 to n . For each cell j ($1 \leq j \leq n$), there is a dish with a deliciousness denoted by an integer a_j . Note that the food's deliciousness can be negative, meaning that Danh dislikes the food (such as bitter vegetables or strict diet food), which will decrease his happiness.

Danh leaps whenever he is happy, and today he is increasingly happy, thus his leap distance increases over time! He starts his journey at time step $i = 1$. At each time step $i > 0$, if Danh is currently at cell p , his growing happiness compels him to hop forward by either i or $i + 1$ steps. This means his next destination will be either cell $p + i$ or cell $p + i + 1$.

Whenever Danh lands on a cell $j \leq n$, he must eat the dish in that cell, which changes his total happiness by a_j . If a chosen jump takes him beyond cell n (i.e., his destination coordinate is strictly greater than n), he happily jumps off the board, and his journey ends immediately. Danh cannot stop his journey early; he must keep jumping until he lands outside the board.

Find the maximum total happiness Danh the happy pig can attain before his journey ends.

Note: Due to the large size of the input, using fast I/O methods is highly recommended to solve this problem.

Input

- The first line contains a single integer n ($1 \leq n \leq 5 \cdot 10^6$) — the index of the last cell on the board.
- The second line contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_j \leq 10^9$) — the deliciousness of the food in each cell from 1 to n .

Output

Output a single integer — the maximum happiness Danh the happy pig can attain.

Sample Input	Sample Output
5 5 -2 8 0 -9	13



Problem J

Crafting Costs

Time limit: 3s
Memory limit: 256MB

An electronics workshop sorts the parts in its warehouse into n kinds of components, numbered from 1 to n . Every kind is sold at the market: one component of kind i costs p_i , and the workshop can buy as many components of each kind as it needs.

Besides buying, the workshop can assemble components itself, following m recipes written in its handbook. Recipe j lists k_j pairwise distinct kinds, all different from kind t_j : take one component of each listed kind, fit them together and pay a labor fee of c_j , and the result is one component of kind t_j . A recipe is only a note in the handbook, so it can be used any number of times. The components that are fitted together are used up in the product. Therefore, if two places in the whole process of obtaining a component both need a component of kind X , the workshop must provide two components of kind X , each obtained separately; one component cannot serve both places.

A kind may be an input of several recipes and, at the same time, the result of several other recipes. The handbook also describes how to take large assemblies apart to get smaller parts, so it may well happen that obtaining kind A requires kind B while obtaining kind B requires kind A .

For every kind of component, find the smallest amount of money the workshop has to spend to obtain one component of that kind after a finite number of purchases and assemblies.

Input

- The first line contains two integers n and m ($1 \leq n \leq 2 \cdot 10^5$, $0 \leq m \leq 2 \cdot 10^5$), the number of kinds and the number of recipes.
- The second line contains n integers $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq 10^9$), the market prices.
- The j -th of the next m lines describes recipe j . It contains three integers t_j, c_j, k_j ($1 \leq t_j \leq n$, $0 \leq c_j \leq 10^9$, $1 \leq k_j \leq n - 1$) followed by k_j integers, the kinds of the inputs of the recipe. These k_j kinds are pairwise distinct, lie between 1 and n , and none of them equals t_j .
- The sum of k_j over all recipes does not exceed $5 \cdot 10^5$.

The input can be almost 10 MB in size, so use a fast way of reading it.

Output

Print one line with n integers. The i -th of them must be the smallest amount of money needed to obtain one component of kind i .



Sample Input	Sample Output
5 4 100 50 40 90 10 1 5 2 2 3 2 20 1 5 4 3 1 1 3 1 1 4	75 30 40 78 10
4 0 7 3 9 2	7 3 9 2
3 2 100 50 6 1 0 2 2 3 2 0 1 3	12 6 6

Notes

In the first example, a component of kind 2 is assembled from a component of kind 5 for $20 + 10 = 30$, a component of kind 1 from components of kinds 2 and 3 for $5 + 30 + 40 = 75$, and a component of kind 4 from a component of kind 1 for $3 + 75 = 78$. The last recipe would make kind 3 from kind 4 for $1 + 78 = 79$, which is more than the market price 40.

In the third example, a component of kind 2 is assembled from a component of kind 3 for $0 + 6 = 6$. Assembling kind 1 needs one component of kind 2 and one component of kind 3. The component of kind 2 itself uses up a component of kind 3, so two components of kind 3 are needed in total, and the cost of kind 1 is $0 + 6 + 6 = 12$, not 6.



Problem K

Lithium and Lithuania

Time limit: 1s
Memory limit: 1GB

Lithium is planning to study abroad in Lithuania, and she needs an IELTS certificate fast. To achieve her target score, she has taken the IELTS test T times.

For each test, she receives four individual component scores: Listening, Reading, Writing, and Speaking. Each component score is a number between 0.0 and 9.0, ending in either .0 or .5.

The overall IELTS score is calculated as the average of the four component scores, rounded to the nearest whole or half band. To avoid ambiguity, the specific rounding rules are defined as follows:

- The average of the four scores is taken.
- The result is rounded to the nearest multiple of 0.5.
- If the average is exactly halfway between two multiples of 0.5 (i.e., its fractional part is .25 or .75), it is rounded **up** to the higher multiple of 0.5.

For example:

- An average of 6.25 is rounded up to 6.5.
- An average of 6.75 is rounded up to 7.0.
- An average of 6.125 is rounded down to 6.0.
- An average of 6.875 is rounded up to 7.0.

Given the four component scores of each of Lithium's T tests, calculate the overall IELTS score for each test.

Input

- The first line contains a single integer T ($1 \leq T \leq 10^4$) — the number of times Lithium took the test.
- Each of the next T lines contains four real numbers L, R, W, S ($0.0 \leq L, R, W, S \leq 9.0$) — the Listening, Reading, Writing, and Speaking scores, respectively. Each number is formatted with exactly one decimal place, which is either 0 or 5.

Output

Print T lines. The i -th line should contain the overall IELTS score of the i -th test, formatted with exactly one decimal place (e.g., 7.0, 6.5).

Sample Input	Sample Output
2	7.0
5.0 5.0 9.0 9.0	7.5
6.0 6.0 9.0 9.0	



Notes

Consider the following sample query:

5.0 5.0 9.0 9.0

The sum of the four components is $5.0 + 5.0 + 9.0 + 9.0 = 28.0$. The average is $\frac{28.0}{4} = 7.0$. Thus, the overall score is 7.0.



Problem L

Lucky Numbers

Time limit: 1s

Memory limit: 256MB

Binh is fond of numbers that are divisible by 3. He believes they bring good luck, so he calls every natural number divisible by 3 a *lucky number*. Natural numbers here start from 0, so 0 is a lucky number too.

One day Binh made up a game for his friends: write as many lucky numbers as possible on a sheet of paper. The catch is that the digits are limited. In total, the digit 0 may be used at most d_0 times, the digit 1 at most d_1 times, and so on, up to the digit 9, which may be used at most d_9 times. The same lucky number may be written any number of times.

Every number is written in the usual decimal notation, without leading zeros. It is not necessary to use all the available digits.

Help Binh's friends: find the largest number of lucky numbers that can be written with the available digits.

Input

- The only line contains ten integers $d_0, d_1, \dots, d_9$ ($0 \leq d_i \leq 10^6$), where d_i is the largest number of times the digit i may be used.

Output

Print a single integer, the largest number of lucky numbers that can be written.

Sample Input	Sample Output
1 1 1 0 0 3 0 0 0 1	4
0 1 0 0 1 0 0 0 0 0	0
2 4 0 1 3 1 0 0 2 0	7

Notes

In the first example, one can write the four numbers 0, 9, 12 and 555.

In the second example, only the digits 1 and 4 are available, and none of the numbers 1, 4, 14, 41 is divisible by 3.